

Assignment-13

HT no: 2303A51694

Name: P.Niharika

Batch-24

Task Description #1 (Refactoring – Removing Code Duplication) •

Task: Use AI to refactor a given Python script that contains multiple repeated code blocks.

- Instructions:
 - o Prompt AI to identify duplicate logic and replace it with functions or classes.
 - o Ensure the refactored code maintains the same output.
 - o Add docstrings to all functions.
- Sample Legacy Code:

```
# Legacy script with repeated logic
print("Area of Rectangle:", 5 * 10)
print("Perimeter of Rectangle:", 2 * (5 + 10))
print("Area of Rectangle:", 7 * 12)
print("Perimeter of Rectangle:", 2 * (7 + 12))
print("Area of Rectangle:", 10 * 15)
print("Perimeter of Rectangle:", 2 * (10 + 15))
```

- Expected Output:
 - o Refactored code with a reusable function and no duplication.
 - o Well documented code

```

ass_13.py > ...
1  #task-1
2  def calculate_rectangle_properties(length, width):
3      """
4      Calculate and print the area and perimeter of a rectangle.
5      Parameters:
6      length (int or float): The length of the rectangle.
7      width (int or float): The width of the rectangle.
8      Returns:
9      None: This function prints the area and perimeter.
10     """
11     area = length * width
12     perimeter = 2 * (length + width)
13     print("Area of Rectangle:", area)
14     print("Perimeter of Rectangle:", perimeter)
15 def main():
16     """
17     Main function to execute rectangle calculations
18     for multiple sets of dimensions.
19
20     Returns:
21     None
22     """
23     rectangles = [(5, 10), (7, 12), (10, 15)]
24     for length, width in rectangles:
25         calculate_rectangle_properties(length, width)
26 if __name__ == "__main__":
27     main()

```

```

PS C:\Users\2303a\OneDrive\Desktop\AI> & C:\Users\2303a\miniconda3\python.exe c:/Users/2303a/OneDrive/Desktop/AI/ass_13.py
Area of Rectangle: 50
Perimeter of Rectangle: 30
Perimeter of Rectangle: 30
Area of Rectangle: 84
Perimeter of Rectangle: 38
Area of Rectangle: 150
Perimeter of Rectangle: 50
PS C:\Users\2303a\OneDrive\Desktop\AI>

```

Task Description #2 (Refactoring – Extracting Reusable Functions) •

Task: Use AI to refactor a legacy script where multiple calculations are embedded directly inside the main code block.

- Instructions: o Identify repeated or related logic and extract it into reusable functions.

o Ensure the refactored code is modular, easy to read, and documented with docstrings.

- Sample Legacy Code:

Week7

-

Monda

y

Legacy script with inline repeated logic

```
price = 250 tax = price * 0.18 total =
```

```
price + tax print("Total Price:", total)
```

```
price = 500 tax = price * 0.18 total =
```

```
price + tax print("Total Price:", total)
```

- Expected Output: o Code with a function

calculate_total(price) that can be reused for multiple price

inputs. o Well documented code

```

#task-2
def calculate_total(price):
    """
    Calculate the total price including 18% tax.
    Parameters:
    price (int or float): The original price of the product
    Returns:
    float: The total price after adding 18% tax.
    """
    tax = price * 0.18
    total = price + tax
    return total

def main():
    """
    Main function to calculate and print total prices
    for multiple product values.
    Returns:
    None
    """
    prices = [250, 500]
    for price in prices:
        total = calculate_total(price)
        print("Total Price:", total)

if __name__ == "__main__":
    main()

```

```

Total Price: 590.0
PS C:\Users\2303a\OneDrive\Desktop\AI> & C:\Users\2303a\miniconda3\python.exe c:/Users/2303a/OneDrive/Desktop/AI/ass_13.py
Total Price: 295.0
Total Price: 590.0

```

Task Description #3: Refactoring Using Classes and Methods (Eliminating Redundant Conditional Logic)

Refactor a Python script that contains repeated if–elif–else grading logic by implementing a structured, object-oriented solution using a class and a method.

Problem Statement

The given script contains duplicated conditional statements used to assign grades based on student marks. This redundancy violates clean code principles and reduces maintainability.

You are required to refactor the script using a class-based design to improve modularity, reusability, and readability while preserving the original grading logic.

Mandatory Implementation Requirements

1. Class Name: GradeCalculator
2. Method Name: calculate_grade(self, marks)
3. The method must:
 - o Accept marks as a parameter.
 - o Return the corresponding grade as a string.
 - o The grading logic must strictly follow the conditions below:
 - Marks ≥ 90 and $\leq 100 \rightarrow$ "Grade A"
 - Marks $\geq 80 \rightarrow$ "Grade B"
 - Marks $\geq 70 \rightarrow$ "Grade C"
 - Marks $\geq 40 \rightarrow$ "Grade D"
 - Marks $\geq 0 \rightarrow$ "Fail"

Note: Assume marks are within the valid range of 0 to 100.

4. Include proper docstrings for:
 - o The class
 - o The method (with parameter and return descriptions)
 5. The method must be reusable and called multiple times without rewriting conditional logic.
- Given code:

```
marks = 85 if
marks >= 90:
print("Grade A")
elif marks >= 75:
print("Grade B")
else:
print("Grade C")

marks = 72 if
marks >= 90:
print("Grade A")
elif marks >= 75:
print("Grade B")
else:
print("Grade C")
```

Expected Output:

- Define a class named GradeCalculator.
- Implement a method calculate_grade(self, marks) inside the class.
- Create an object of the class.
- Call the method for different student marks.
- Print the returned grade values.

```
ass_13.py > ...
1  #task-3
2  class GradeCalculator:
3      """
4      A class used to calculate student grades based on marks.
5      Methods:
6      calculate_grade(self, marks):
7          Returns the grade string based on the given marks.
8      """
9      def calculate_grade(self, marks):
10         """
11         Determine the grade for a given mark.
12         Parameters:
13         marks (int or float): The student's marks (assumed between 0 and 100).
14         Returns:
15         str: The grade corresponding to the marks based on the grading rules.
16         """
17         if marks >= 90 and marks <= 100:
18             return "Grade A"
19         elif marks >= 80:
20             return "Grade B"
21         elif marks >= 70:
22             return "Grade C"
23         elif marks >= 40:
24             return "Grade D"
25         elif marks >= 0:
26             return "Fail"
27     def main():
28         """
29         Main function to demonstrate reusable grade calculation.
30         """
31         calculator = GradeCalculator()
32         marks_list = [85, 72, 95, 38]
33         for marks in marks_list:
34             grade = calculator.calculate_grade(marks)
35             print(f"Marks: {marks} -> {grade}")
36 if __name__ == "__main__":
37     main()
```

```
PS C:\Users\2303a\OneDrive\Desktop\AI> & C:\Users\2303a\miniconda3\python.exe c:/Users/2303a/OneDrive/Desktop/AI/ass_13.py
Marks: 85 -> Grade B
Marks: 72 -> Grade C
Marks: 95 -> Grade A
Marks: 38 -> Fail
PS C:\Users\2303a\OneDrive\Desktop\AI> □
```

Task Description #4 (Refactoring – Converting Procedural Code to Functions)

- Task: Use AI to refactor procedural input–processing logic into functions.

Instructions:

o Identify input, processing, and output sections. o

Convert each into a separate function. o Improve code

readability without changing behavior.

- Sample Legacy Code: `num = int(input("Enter`

`number: ")) square = num * num`

`print("Square:", square)`

- Expected Output: o Modular code using functions like `get_input()`, `calculate_square()`, and `display_result()`.

```
ass_13.py > ...
1  #task-4
2  def get_input():
3      """
4      Prompt the user to enter a number.
5      Returns:
6      int: The number entered by the user.
7      """
8      return int(input("Enter number: "))
9  def calculate_square(num):
10     """
11     Calculate the square of a number.
12     Parameters:
13     num (int or float): The number to be squared.
14     Returns:
15     int or float: The square of the given number.
16     """
17     return num * num
18  def display_result(square):
19     """
20     Display the square result.
21     Parameters:
22     square (int or float): The calculated square value.
23     Returns:
24     None
25     """
26     print("Square:", square)
27  def main():
28     """
29     Main function to coordinate input, processing,
30     and output operations.
31     """
32     number = get_input()
33     square = calculate_square(number)
34     display_result(square)
35  if __name__ == "__main__":
36     main()
```

```
PS C:\Users\2303a\OneDrive\Desktop\AI> & C:\Users\2303a\miniconda3\python.exe c:/Users/2303a/OneDrive/Desktop/AI/ass_13.py
Enter number: 10
Square: 100
PS C:\Users\2303a\OneDrive\Desktop\AI> 
```


Task 5 (Refactoring Procedural Code into OOP Design)

- Task: Use AI to refactor procedural code into a class-based design.

Focus Areas: o Object-

Oriented principles o

Encapsulation Legacy Code:

```
salary = 50000 tax = salary
```

```
* 0.2 net = salary - tax
```

```
print(net)
```

Expected Outcome:

- o A class like EmployeeSalaryCalculator with methods and attributes.

```

ass_13.py > ...
1  #task-5
2  class EmployeeSalaryCalculator:
3      """
4      A class to calculate an employee's net salary after tax deduction.
5      Attributes:
6      salary (float): The employee's gross salary.
7      tax_rate (float): The tax percentage applied to the salary.
8      """
9      def __init__(self, salary, tax_rate=0.2):
10         """
11         Initialize the EmployeeSalaryCalculator with salary and tax rate.
12         Parameters:
13         salary (float): The employee's gross salary.
14         tax_rate (float, optional): The tax rate (default is 0.2 or 20%).
15         """
16         self.salary = salary
17         self.tax_rate = tax_rate
18     def calculate_tax(self):
19         """
20         Calculate the tax amount based on salary and tax rate.
21         Returns:
22         float: The calculated tax amount.
23         """
24         return self.salary * self.tax_rate
25     def calculate_net_salary(self):
26         """
27         Calculate the net salary after tax deduction.
28         Returns:
29         float: The net salary.
30         """
31         tax = self.calculate_tax()
32         return self.salary - tax
33     def display_net_salary(self):
34         """
35         Display the net salary.
36         Returns:
37         None

```

```

37         None
38         """
39         net_salary = self.calculate_net_salary()
40         print("Net Salary:", net_salary)
41     def main():
42         """
43         Main function to demonstrate salary calculation.
44         """
45         employee = EmployeeSalaryCalculator(50000)
46         employee.display_net_salary()
47     if __name__ == "__main__":
48         main()
49

```

```

PS C:\Users\2303a\OneDrive\Desktop\AI> & C:\Users\2303a\miniconda3\python.exe c:/Users/2303a/OneDrive/Desktop/AI/ass_13.py
Net Salary: 40000.0
PS C:\Users\2303a\OneDrive\Desktop\AI>

```

Task 6 (Optimizing Search Logic)

- Task: Refactor inefficient linear searches

using appropriate data structures.

- Focus Areas: o Time complexity o Data

structure choice

Legacy Code:

```
users = ["admin", "guest", "editor", "viewer"] name =
```

```
input("Enter username: ") found = False for u in
```

```
users: if u == name: found = True print("Access
```

```
Granted" if found else "Access Denied")
```

Expected Outcome:

- o Use of sets or dictionaries with complexity justification

```
ass_13.py > ...
1  #task-6
2  def check_access(users_set, username):
3      """
4      Check whether the given username exists in the system.
5      Parameters:
6      users_set (set): A set of valid usernames.
7      username (str): The username to verify.
8      Returns:
9      bool: True if user exists, otherwise False.
10     """
11     return username in users_set
12 def main():
13     """
14     Main function to handle user input and access verification.
15     """
16     users = {"admin", "guest", "editor", "viewer"} # Using set for O(1) lookup
17     name = input("Enter username: ")
18     if check_access(users, name):
19         print("Access Granted")
20     else:
21         print("Access Denied")
22 if __name__ == "__main__":
23     main()
24
```

```
PS C:\Users\2303a\OneDrive\Desktop\AI> & C:\Users\2303a\miniconda3\python.exe c:/Users/2303a/OneDrive/Desktop/AI/ass_13.py
Enter username: charanya
Access Denied
PS C:\Users\2303a\OneDrive\Desktop\AI> |
```

Task 7 – Refactoring the Library Management System

Problem Statement

You are provided with a poorly structured Library Management script that:

- Contains repeated conditional logic
- Does not use reusable functions
- Lacks documentation
- Uses print-based procedural execution
- Does not follow modular programming principles

Your task is to refactor the code into a proper format

1. Create a module library.py with functions:
 - o add_book(title, author, isbn)
 - o remove_book(isbn)
 - o search_book(isbn)
2. Insert triple quotes under each function and let Copilot complete the docstrings.
3. Generate documentation in the terminal.
4. Export the documentation in HTML format.
5. Open the file in a browser.

Given Code

```
# Library Management System (Unstructured Version)

# This code needs refactoring into a proper module with documentation.

library_db = {} #

Adding first book title

= "Python Basics"

author = "John Doe"

isbn = "101" if isbn not

in library_db:

library_db[isbn] = {"title": title, "author": author}

print("Book added successfully.") else:

print("Book already exists.")
```

```

# Adding second book (duplicate logic)

title = "AI Fundamentals" author =

"Jane Smith" isbn = "102" if isbn not in

library_db:

library_db[isbn] = {"title": title, "author": author}

print("Book added successfully.") else:

print("Book already exists.") # Searching

book (repeated logic structure) isbn =

"101" if isbn in library_db:

print("Book Found:", library_db[isbn]) else:

print("Book not found.")

# Removing book (again repeated pattern)

isbn = "101" if isbn in library_db: del

library_db[isbn] print("Book removed

successfully.") else:

print("Book not found.")

# Searching again

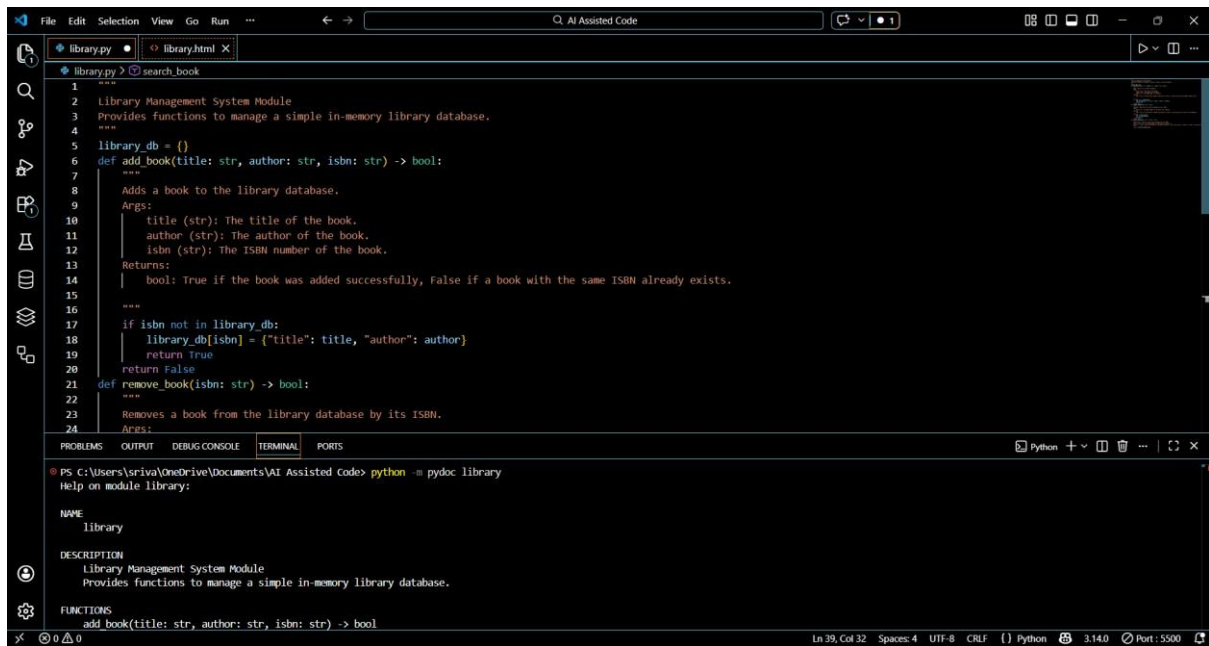
isbn = "101" if isbn

in library_db:

print("Book Found:", library_db[isbn])

else: print("Book not found.")

```



```
1 """
2 Library Management System Module
3 Provides functions to manage a simple in-memory library database.
4 """
5 library_db = {}
6 def add_book(title: str, author: str, isbn: str) -> bool:
7     """
8     Adds a book to the library database.
9     Args:
10         title (str): The title of the book.
11         author (str): The author of the book.
12         isbn (str): The ISBN number of the book.
13     Returns:
14         bool: True if the book was added successfully, False if a book with the same ISBN already exists.
15     """
16     if isbn not in library_db:
17         library_db[isbn] = {"title": title, "author": author}
18         return True
19     return False
20 def remove_book(isbn: str) -> bool:
21     """
22     Removes a book from the library database by its ISBN.
23     Args:
24         isbn (str): The ISBN number of the book to be removed.
25     Returns:
26         bool: True if the book was removed successfully, False if the book was not found in the database.
27     """
28     if isbn in library_db:
29         del library_db[isbn]
30         return True
31     return False
32 def search_book(isbn: str) -> dict | None:
33     """
34     Searches for a book in the library database by its ISBN.
35     Args:
36         isbn (str): The ISBN number of the book to search for.
37     Returns:
38         dict | None: A dictionary containing the book's title and author if found, or None if the book is not found in the database.
39     """
40     if isbn in library_db:
41         return library_db[isbn]
42     return None
```

[index](#)

library <c:\users\sriva\onedrive\documents\ai assisted code\library.py>

Library Management System Module
Provides functions to manage a simple in-memory library database.

Functions

add_book(title: str, author: str, isbn: str) -> bool
Adds a book to the library database.
Args:
 title (str): The title of the book.
 author (str): The author of the book.
 isbn (str): The ISBN number of the book.
Returns:
 bool: True if the book was added successfully, False if a book with the same ISBN already exists.

remove_book(isbn: str) -> bool
Removes a book from the library database by its ISBN.
Args:
 isbn (str): The ISBN number of the book to be removed.
Returns:
 bool: True if the book was removed successfully, False if the book was not found in the database.

search_book(isbn: str) -> dict | None
Searches for a book in the library database by its ISBN.
Args: isbn (str): The ISBN number of the book to search for.
Returns: dict | None: A dictionary containing the book's title and author if found, or None if the book is not found in the database.

Data

library_db = {}

Task 8– Fibonacci Generator

Write a program to generate Fibonacci series up to n.

The initial code has:

- Global variables.
- Inefficient loop.
- No functions or modularity.

Task for Students:

- Refactor into a clean reusable function (generate_fibonacci).
- Add docstrings and test cases.
- Compare AI-refactored vs original.

Bad Code Version: #

fibonacci bad version

```
n=int(input("Enter limit: "))
```

```
a=0 b=1
```

```
print(a) print(b)
```

```
for i in range(2,n):
```

```
c=a+b
```

```
print(c)
```

```
a=b b=c
```



```

ass_13.py > ...
1  #task-8
2  def generate_fibonacci(n):
3      """
4      Generate Fibonacci series up to n terms.
5      Parameters:
6      n (int): Number of terms to generate. Must be >= 0.
7      Returns:
8      list: A list containing the Fibonacci sequence up to n terms.
9      """
10     if n <= 0:
11         return []
12     if n == 1:
13         return [0]
14     fibonacci_series = [0, 1]
15     for _ in range(2, n):
16         fibonacci_series.append(
17             fibonacci_series[-1] + fibonacci_series[-2]
18         )
19     return fibonacci_series
20 def main():
21     """
22     Main function to demonstrate Fibonacci generation.
23     """
24     n = int(input("Enter limit: "))
25     result = generate_fibonacci(n)
26     print("Fibonacci Series:", result)
27 if __name__ == "__main__":
28     main()

```

```

PS C:\Users\2303a\OneDrive\Desktop\AI> & C:\Users\2303a\miniconda3\python.exe c:/Users/2303a/OneDrive/Desktop/AI/ass_13.py
Enter limit: 10
Fibonacci Series: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
PS C:\Users\2303a\OneDrive\Desktop\AI>

```

Task 9 – Twin Primes Checker

Twin primes are pairs of primes that differ by 2 (e.g., 11 and 13, 17 and 19).

The initial code has:

- Inefficient prime checking.
- No functions.
- Hardcoded inputs.

Task for Students:

- Refactor into `is_prime(n)` and `is_twin_prime(p1, p2)`.
- Add docstrings and optimize.
- Generate a list of twin primes in a given range using AI.

Bad Code Version: # twin

primes bad version a=11

b=13 fa=0

for i in range(2,a):

if a%i==0: fa=1

fb=0

for i in range(2,b): if b%i==0: fb=1 if

fa==0 and fb==0 and abs(a-b)==2:

print("Twin Primes") else:

print("Not Twin Primes")

```
ass_13.py > ...
1  import math
2  def is_prime(n):
3      """
4      Check whether a number is prime using optimized logic.
5      Parameters:
6      n (int):# The number to check.
7      Returns:
8      bool: True if n is prime, otherwise False.
9      """
10     if n <= 1:
11         return False
12     if n <= 3:
13         return True
14     if n % 2 == 0:
15         return False
16     # Check only up to sqrt(n)
17     for i in range(3, int(math.sqrt(n)) + 1, 2):
18         if n % i == 0:
19             return False
20     return True
21 def is_twin_prime(p1, p2):
22     """
23     Check whether two numbers form a twin prime pair.
24     Parameters:
25     p1 (int): First number.
26     p2 (int): Second number.
27     Returns:
28     bool: True if both are prime and differ by 2, otherwise False.
29     """
30     return is_prime(p1) and is_prime(p2) and abs(p1 - p2) == 2
31 def generate_twin_primes(start, end):
32     """
33     Generate all twin prime pairs within a given range.
34     Parameters:
35     start (int): Starting number of the range.
36     end (int): Ending number of the range.
37     Returns:
```

```
PS C:\Users\2303a\OneDrive\Desktop\AI> & C:\Users\2303a\mini.conda3\python.exe c:/Users/2303a/OneDrive/Desktop/AI/ass_13.py
Check specific pair (11, 13):
o Twin Primes

Twin primes between 1 and 50:
[(3, 5), (5, 7), (11, 13), (17, 19), (29, 31), (41, 43)]
PS C:\Users\2303a\OneDrive\Desktop\AI> █
```

Task 10 – Refactoring the Chinese Zodiac Program

Objective

Refactor the given poorly structured Python script into a clean, modular, and reusable implementation.

The current program reads a year from the user and prints the corresponding Chinese Zodiac sign. However, the implementation contains repetitive

conditional logic, lacks modular design, and does not follow clean coding principles.

Your task is to refactor the code to improve readability, maintainability, and structure.

Chinese Zodiac Cycle (Repeats Every 12 Years)

1. Rat
2. Ox
3. Tiger
4. Rabbit
5. Dragon
6. Snake
7. Horse
8. Goat (Sheep)
9. Monkey
10. Rooster
11. Dog
12. Pig

Chinese Zodiac Program (Unstructured Version)

This code needs refactoring.

```
year = int(input("Enter a year: ")) if
```

```
year % 12 == 0: print("Monkey")
```

```
elif year % 12 == 1:

    print("Rooster") elif

year % 12 == 2:

    print("Dog") elif year

% 12 == 3:

    print("Pig") elif year

% 12 == 4:

    print("Rat") elif year

% 12 == 5:

    print("Ox") elif year

% 12 == 6:

    print("Tiger") elif

year % 12 == 7:

    print("Rabbit") elif

year % 12 == 8:

    print("Dragon") elif

year % 12 == 9:

    print("Snake") elif

year % 12 == 10:

    print("Horse") elif

year % 12 == 11:

    print("Goat")
```

You must:

1. Create a reusable function: `get_zodiac(year)`
2. Replace the if-elif chain with a cleaner structure (e.g., list or dictionary).
3. Add proper docstrings.
4. Separate input handling from logic.
5. Improve readability and maintainability.
6. Ensure output remains correct.

```
ass_13.py > ...
1  #task-10
2  def get_zodiac(year):
3      """
4      Determine the Chinese Zodiac sign for a given year.
5      Parameters:
6      year (int): The year for which the zodiac sign is required.
7      Returns:
8      str: The corresponding Chinese Zodiac sign.
9      """
10     zodiac_cycle = [
11         "Monkey", # 0
12         "Rooster", # 1
13         "Dog", # 2
14         "Pig", # 3
15         "Rat", # 4
16         "Ox", # 5
17         "Tiger", # 6
18         "Rabbit", # 7
19         "Dragon", # 8
20         "Snake", # 9
21         "Horse", # 10
22         "Goat" # 11
23     ]
24     return zodiac_cycle[year % 12]
25 def main():
26     """
27     Handle user input and display the zodiac sign.
28     """
29     year = int(input("Enter a year: "))
30     zodiac_sign = get_zodiac(year)
31     print(zodiac_sign)
32 if __name__ == "__main__":
33     main()
34
```

PS C:\Users\2303a\OneDrive\Desktop\AI> & C:\Users\2303a\miniconda3\python.exe c:/Users/2303a/OneDrive/Desktop/AI/ass_13.py
Enter a year: 2004
Monkey

Task 11 – Refactoring the Harshad (Niven) Number Checker

Refactor the given poorly structured Python script into a clean, modular, and reusable implementation.

A Harshad (Niven) number is a number that is divisible by the sum of its digits.

For example:

- $18 \rightarrow 1 + 8 = 9 \rightarrow 18 \div 9 = 2$ (Harshad Number)
- $19 \rightarrow 1 + 9 = 10 \rightarrow 19 \div 10 \neq \text{integer}$ (Not Harshad)

Problem Statement

The current implementation:

- Mixes logic and input handling
- Uses redundant variables
- Does not use reusable functions properly
- Returns print statements instead of boolean values
- Lacks documentation

You must refactor the code to follow clean coding principles.

```
# Harshad Number Checker (Unstructured Version) num =
```

```
int(input("Enter a number: ")) temp = num sum_digits = 0
```

```
while temp > 0: digit = temp
```

```
% 10 sum_digits = sum_digits
```

```
+ digit temp = temp // 10 if
```

```
sum_digits != 0: if num %
```

```
sum_digits == 0:
```

```
print("True") else:
```

```
print("False") else:
```

```
print("False")
```

You must:

1. Create a reusable function: `is_harshad(number)`
2. The function must:
 - o Accept an integer parameter.
 - o Return True if the number is divisible by the sum of its digits.
 - o Return False otherwise.
3. Separate user input from core logic.
4. Add proper docstrings.
5. Improve readability and maintainability.
6. Ensure the program handles edge cases (e.g., 0, negative numbers).


```

ass_13.py > ...
1  #task-11
2  def is_harshad(number):
3      """
4      Determine whether a number is a Harshad (Niven) number.
5      A Harshad number is divisible by the sum of its digits.
6      Parameters:
7      number (int): The integer to check.
8      Returns:
9      bool: True if the number is a Harshad number,
10         | False otherwise.
11      Edge Cases:
12      - Returns False for 0 (division undefined).
13      - Works with negative numbers by using absolute value.
14      """
15      if number == 0:
16          return False
17      num = abs(number)
18      digit_sum = sum(int(digit) for digit in str(num))
19      if digit_sum == 0:
20          return False
21      return num % digit_sum == 0
22  def main():
23      """
24      Handle user input and display whether the number
25      is a Harshad number.
26      """
27      number = int(input("Enter a number: "))
28      result = is_harshad(number)
29      print(result)
30  if __name__ == "__main__":
31      main()

PS C:\Users\2303a\OneDrive\Desktop\AI> & C:\Users\2303a\miniconda3\python.exe c:/Users/2303a/OneDrive/Desktop/AI/ass_13.py
Enter a number: 15
False
PS C:\Users\2303a\OneDrive\Desktop\AI>

```

Task 12 – Refactoring the Factorial Trailing Zeros Program

Refactor the given poorly structured Python script into a clean, modular, and efficient implementation.

The program calculates the number of trailing zeros in $n!$ (factorial of n).

Problem Statement

The current implementation:

- Calculates the full factorial (inefficient for large n)
- Mixes input handling with business logic
- Uses print statements instead of return values
- Lacks modular structure and documentation

You must refactor the code to improve efficiency, readability, and maintainability.

Factorial Trailing Zeros (Unstructured Version)

```
n = int(input("Enter a number: ")) fact = 1 i = 1
while i <= n: fact = fact * i i = i + 1 count = 0
while fact % 10 == 0: count = count + 1 fact =
fact // 10 print("Trailing zeros:", count)
```

You must:

1. Create a reusable function: `count_trailing_zeros(n)`
2. The function must:
 - o Accept a non-negative integer n.
 - o Return the number of trailing zeros in n!.
3. Do NOT compute the full factorial.
4. Use an optimized mathematical approach (count multiples of 5).
5. Add proper docstrings.
6. Separate user interaction from core logic.
7. Handle edge cases (e.g., negative numbers, zero).

Test Cases Design


```

ass_13.py > ...
1  #task-12
2  def count_trailing_zeros(n):
3      """
4      Calculate the number of trailing zeros in n! (factorial of n).
5      The function uses an optimized mathematical approach by
6      counting multiples of 5 instead of computing the full factorial.
7      Parameters:
8      n (int): A non-negative integer.
9      Returns:
10     int: The number of trailing zeros in n!.
11     Raises:
12     ValueError: If n is negative.
13     """
14     if n < 0:
15         raise ValueError("Input must be a non-negative integer.")
16     count = 0
17     divisor = 5
18     while n // divisor > 0:
19         count += n // divisor
20         divisor *= 5
21     return count
22 def main():
23     """
24     Handle user input and display trailing zeros result.
25     """
26     n = int(input("Enter a number: "))
27     try:
28         result = count_trailing_zeros(n)
29         print("Trailing zeros:", result)
30     except ValueError as e:
31         print(e)
32 if __name__ == "__main__":
33     main()

```

PS C:\Users\2303a\OneDrive\Desktop\AI> & C:\Users\2303a\miniconda3\python.exe c:/Users/2303a/OneDrive/Desktop/AI/ass_13.py
Enter a number: 12
Trailing zeros: 2
PS C:\Users\2303a\OneDrive\Desktop\AI> █

Task 13 (Collatz Sequence Generator – Test Case Design)

- Function: Generate Collatz sequence until reaching 1.
- Test Cases to Design:
- Normal: $6 \rightarrow [6, 3, 10, 5, 16, 8, 4, 2, 1]$
- Edge: $1 \rightarrow [1]$

- Negative: -5
- Large: 27 (well-known long sequence) • Requirement: Validate correctness with pytest.

Explanation:

We need to write a function that:

- Takes an integer n as input.
- Generates the Collatz sequence (also called the $3n+1$ sequence).
- The rules are:

o If n is even $\rightarrow$ next = $n / 2$. o

If n is odd $\rightarrow$ next = $3n + 1$.

- Repeat until we reach 1.
- Return the full sequence as a list.

Example

Input: 6

Steps:

- 6 (even $\rightarrow 6/2 = 3$)
- 3 (odd $\rightarrow 3*3+1 = 10$)
- 10 (even $\rightarrow 10/2 = 5$)
- 5 (odd $\rightarrow 3*5+1 = 16$)
- 16 (even $\rightarrow 16/2 = 8$)
- 8 (even $\rightarrow 8/2 = 4$)
- 4 (even $\rightarrow 4/2 = 2$)

- 2 (even $\rightarrow 2/2 = 1$) Output:

[6, 3, 10, 5, 16, 8, 4, 2, 1]

The screenshot shows a code editor with two files: `lab.py` and `test_lab.py`. The `lab.py` file contains the `collatz_sequence` function, which takes an integer `n` and returns a list of integers representing the Collatz sequence. The function includes a check for positive integers and a loop that generates the sequence by dividing even numbers by 2 and multiplying odd numbers by 3 and adding 1. The `test_lab.py` file contains four test functions: `test_normal_case` (asserts the sequence for `n=6` is [6, 3, 10, 5, 16, 8, 4, 2, 1]), `test_edge_case_one` (asserts the sequence for `n=1` is [1]), `test_negative_input` (uses `pytest.raises` to assert a `ValueError` is raised for `n=-5`), and `test_large_case_27` (calculates the sequence for `n=27` and checks its length and specific elements). The terminal at the bottom shows the command `python -m pytest` being executed, resulting in 4 passed tests in 0.06s.

```

lab.py
1 def collatz_sequence(n: int) -> list[int]:
2     if n <= 0:
3         raise ValueError("n must be a positive integer")
4
5     sequence = [n]
6
7     while n != 1:
8         if n % 2 == 0:
9             n = n // 2
10        else:
11            n = 3 * n + 1
12        sequence.append(n)
13
14    return sequence

test_lab.py
5 def test_normal_case():
6     | assert collatz_sequence(6) == [6, 3, 10, 5, 16, 8, 4, 2, 1]
7
8
9 def test_edge_case_one():
10    | assert collatz_sequence(1) == [1]
11
12
13 def test_negative_input():
14     with pytest.raises(ValueError):
15         collatz_sequence(-5)
16
17
18 def test_large_case_27():
19     result = collatz_sequence(27)
20
21     # Basic correctness checks
22     assert result[0] == 27
23     assert result[-1] == 1
24     assert len(result) > 100 # 27 produces a long sequence (112 terms)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\sriva\OneDrive\Documents\AI Assisted Code> python -m pytest
PS C:\Users\sriva\OneDrive\Documents\AI Assisted Code> pytest
platform win32 -- python 3.14.0, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\sriva\OneDrive\Documents\AI Assisted Code
collected 4 items

test_lab.py ....

===== test session starts =====
4 passed in 0.06s

```

Task 14 (Lucas Number Sequence – Test Case Design)

- Function: Generate Lucas sequence up to `n` terms.

(Starts with 2,1, then $F_n = F_{n-1} + F_{n-2}$)

- Test Cases to Design:
- Normal: 5 $\rightarrow$ [2, 1, 3, 4, 7]
- Edge: 1 $\rightarrow$ [2]
- Negative: -5 $\rightarrow$ Error
- Large: 10 (last element = 76).
- Requirement: Validate correctness with pytest.

```
lab.py
1 def lucas_sequence(n: int) -> list[int]:
2     if n < 0:
3         raise ValueError("n must be non-negative")
4
5     if n == 0:
6         return []
7
8     if n == 1:
9         return [2]
10
11     sequence = [2, 1]
12
13     for _ in range(2, n):
14         sequence.append(sequence[-1] + sequence[-2])
15
16     return sequence

test_lab.py
1 from lab import lucas_sequence
2 import pytest
3
4 def test_normal_case():
5     | assert lucas_sequence(5) == [2, 1, 3, 4, 7]
6
7 def test_edge_case_one():
8     | assert lucas_sequence(1) == [2]
9
10 def test_negative_input():
11     | with pytest.raises(ValueError):
12         | lucas_sequence(-5)
13
14 def test_large_case():
15     | result = lucas_sequence(10)
16     | assert len(result) == 10
17     | assert result[-1] == 76

PS C:\Users\sriya\OneDrive\Documents\AI Assisted Code> python -m pytest
PS C:\Users\sriya\OneDrive\Documents\AI Assisted Code> python -m pytest

platform win32 -- Python 3.14.0, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\sriya\OneDrive\Documents\AI Assisted Code
collected 4 items

test_lab.py ....

===== 4 passed in 0.08s =====
```

Task 15 (Vowel & Consonant Counter – Test Case Design)

- Function: Count vowels and consonants in string.
- Test Cases to Design:
- Normal: "hello" → (2,3)
- Edge: "" → (0,0)
- Only vowels: "aeiou" → (5,0)

Large: Long text

- Requirement: Validate correctness with pytest.

